

The Islamic University of Gaza
Faculty of Information Technology
Department of Information Technology
Web Computing, Software
Development, and Mobile Computing
Programs



الجامعة الإسلامية بغزة
كلية تكنولوجيا المعلومات
قسم علم الحاسوب , حوسبة ويب , حوسبة متنقلة

TechVerse Electronic Store - Web-Based E-Commerce Platform

منصة تك فيرس الإلكترونية - منصة تجارة إلكترونية ويب

By

Abdulrahman Ahmed Mohammed Al-Dimassi - 120222041

Momin Ahmed Rezg Alwawi - 120220143

Abdulrahman Alaa Nazmi Al-Jadili - 120220456

Supervisor

Dr. Hazem A. Alrakhawi

A graduation project report submitted in partial fulfillment of the requirements for the degree of
Bachelor of Information Technology

July/2026

Abstract

This research addresses the need for a structured web-based e-commerce platform for electronic devices, especially for local stores in Palestine that still depend on social media, phone calls, or manual ordering processes. These methods make it difficult for customers to compare products, verify specifications, manage carts, and track orders in a reliable way.

The objective of this project is to design and implement TechVerse, a responsive web platform that supports product browsing, search, filtering, favorites, cart management, checkout, and order history. The system was developed using the Agile Scrum methodology to divide the work into iterative sprints, allowing requirements, implementation, testing, and deployment to be improved gradually.

The implementation combines Vue 3, Pinia, Vue Router, Bootstrap, Firebase Realtime Database, plain PHP, MySQL, and Stripe Checkout. Firebase is used for product and user-related data, MySQL stores Stripe order records, and Stripe handles secure online payment. During development and testing, the Stripe Sandbox (test environment) was used to simulate payment transactions without processing real financial operations. Testing results showed that the main workflows passed successfully, including registration, login, product browsing, non-Stripe orders, Stripe payment confirmation, order cancellation, and purchase history. Future recommendations include adding an admin panel, Arabic language support, delivery tracking, analytics, and native mobile applications.

ملخص الدراسة

متخصصة في بيع الأجهزة الإلكترونية، وذلك TechVerse يتناول هذا البحث تصميم وتطوير منصة ويب للتجارة الإلكترونية باسم استجابةً للحاجة إلى نظام منظم يساعد المتاجر المحلية في فلسطين على عرض المنتجات وإدارة الطلبات بطريقة أوضح وأكثر موثوقية بدلاً من الاعتماد على وسائل التواصل الاجتماعي أو الطلبات اليدوية. تتمثل المشكلة في صعوبة وصول الزبائن إلى معلومات المنتجات ومقارنتها وإتمام الشراء وتتبع الطلبات من مكان واحد.

يهدف البحث إلى بناء منصة ويب متجاوبة تُمكن المستخدم من تصفح المنتجات والبحث عنها وتصفيتها، وإضافتها إلى سلة في Agile Scrum المشتريات أو المفضلة، وإتمام عملية الدفع بطرق متعددة، مع إمكانية متابعة سجل المشتريات. تم اعتماد منهجية إدارة العمل من خلال تقسيم المشروع إلى مراحل قصيرة تساعد على التطوير والاختبار والتحسين المستمر.

إضافة إلى Stripe لمعالجة طلبات MySQL مع PHP Plain لتخزين بيانات المنتجات والمستخدمين والمشتريات، و Database Firebase Realtime في الواجهة الأمامية، مع Bootstrap وVue Router وPinia وVue 3 تم تنفيذ النظام باستخدام لتنفيذ الدفع الإلكتروني الآمن. أظهرت نتائج الاختبار نجاح الوظائف الأساسية مثل التسجيل وتسجيل الدخول Stripe Checkout والتصفح والسلة والدفع وتأكيذ الطلبات. وتوصي الدراسة مستقبلاً بإضافة لوحة تحكم للإدارة، ودعم اللغة العربية، وتتبع التوصيل، والتقارير التحليلية، وتطبيقات موبايل مرتبطة بنفس النظام.

Dedication

We dedicate this project to our families, who have given us enough support to our success, To our university, to our supervisor, to our colleagues, to all teachers in Islamic university, to our friends, to everybody help us to do this project.

Acknowledgment

First, we thank Allah for giving us the power to finish this project. We thank our supervisor, Dr. Hazem A. Alrakhawi, for his help, advice, and support during the whole project. We also thank the Islamic University of Gaza and the Faculty of Information Technology for the education and things they gave us. Special thanks to our families and friends for helping us feel good and strong.

The Research Team

Contents

Abstract.....	i
الدراسة ملخص	ii
Dedication.....	iii
Acknowledgment.....	iv
List of Tables.....	ix
List of Figures.....	x
List of Abbreviations.....	xi
Chapter 1	12
Introduction	12
Chapter 1 Introduction.....	1
1.1 Problem Statement.....	1
1.2 Objectives	1
1.2.1 Main Objective	1
1.2.2 Specific Objectives	1
1.3 Scope and Limitations	2
Scope	2
Limitations.....	2
1.4 Importance of the Project	2
1.5 Methodology.....	2
1.6 Tools, Equipment, and Methods.....	3
1.7 Timetable	4
Chapter 2	5
Related Works	5
Chapter 2 Related Works.....	6
2.1 International E-commerce Platforms.....	6
2.1.1 Amazon [13].....	6
2.1.2 Newegg [14].....	6
2.1.3 AliExpress [15].....	7
2.2 Technical Frameworks and Similar Systems	8
Vue 3 and Pinia	8
Firebase Realtime Database	8
Plain PHP REST API	8

Stripe Payment	8
2.3 Comparison with Related Works.....	8
2.4 The Difference Between Our System and Previous Systems.....	8
Chapter 3	10
Requirement Analysis	10
Chapter 3 Requirement Analysis.....	11
3.1 Requirement Collection.....	11
3.2 Actors	11
Guest.....	11
Customer (Registered User)	11
Stripe (External Service)	11
3.3 Functional Requirements.....	11
3.4 Non-functional Requirements	13
3.5 Use Case List.....	13
3.6 Use Case Descriptions.....	14
3.7 Database Structure.....	14
3.8 Database Tables Description	15
3.9 MySQL Schema (Orders Table — Stripe Only)	16
Chapter 4	18
Implementation.....	18
Chapter 4 Implementation	19
4.1 System Architecture	19
4.2 PHP Backend — File Structure.....	19
4.3 config.php.....	20
4.4 db.php — MySQL Connection.....	21
4.5 firebase.php — Reading Product Prices	21
4.6 creatCheckout.php — Stripe Session	22
4.7 webhook.php — Payment Confirmation.....	24
4.8 cancelOrder.php — Cancel an Order	25
4.9 Frontend — Vue.js Pages	25
4.10 Frontend — Pinia Stores	26
4.11 Frontend — Vue Router	26
4.12 Authentication Flow	27

Registration	27
Login	27
4.13 Payment Flows	27
Flow 1 — COD, Credit Card, Bank Transfer.....	27
Flow 2 — Stripe Online Payment	28
Flow 3 — Stripe Cancellation	28
4.14 Security	29
4.15 Deployment	29
Chapter 5	31
User Interface	31
5.1 User Interface	32
Chapter 6	44
Testing	44
Chapter 6 Testing	45
6.1 Unit Testing	45
6.2 Integration Testing.....	45
6.3 Functional Testing	46
6.4 Non-functional Testing.....	46
Usability	46
Performance.....	46
Security	46
Compatibility	47
6.5 Testing Results	47
Chapter 7	48
Conclusion and Future Work.....	48
Chapter 7 Conclusion and Future Work.....	49
7.1 Conclusion.....	49
7.2 Limitations.....	49
7.3 Future Work.....	49
References	50
Appendix	52
Appendix A: Customer Guide	52
Appendix B: Backend Environment Variables	52

Appendix C: Netlify Deployment Fix	53
--	----

List of Tables

Table 1: List of Abbreviations.....	x
Table 2: Tools and Technologies Used in the Project.....	3
Table 3: Agile Scrum Sprint Timeline	4
Table 4: Team Roles and Responsibilities	4
Table 5: Comparison of Related Works by Evaluation Criteria.....	8
Table 6: Functional Requirements.....	11
Table 7: Non-functional Requirements	13
Table 8: Use Case List.....	13
Table 9: Registration Use Case	14
Table 10: Stripe Use Case	14
Table 11: Firebase Realtime Database Schema.....	15
Table 12: MySQL Orders Table Schema	16
Table 13: PHP Backend Endpoints	19
Table 14: Vue.js Pages	25
Table 15: Pinia Stores.....	26
Table 16: Vue Router Routes	27
Table 17: Test Cases.....	45

List of Figures

Figure 1: Agile Scrum model	3
Figure 2: Amazon platform overview	6
Figure 3: Newegg platform overview	7
Figure 4: AliExpress platform overview	7
Figure 5: Firebase Realtime Database structure	15
Figure 6: System Architecture Diagram	19
Figure 7: Hero carousel with categories and featured products.....	30
Figure 8: Product grid with category, brand, and price filters.....	32
Figure 9: Product details with specifications table, add to cart, and favorites	34
Figure 10: Login form with email and password	35
Figure 11: Cart with item list, quantity controls, and order summary	36
Figure 12: Checkout form with shipping info and payment method selection.....	37
Figure 13: Stripe hosted checkout page	38
Figure 14: Success page after Stripe payment confirmed	39
Figure 15: Order history with items, shipping info, and payment details	40
Figure 16: Favorites list with saved products.....	41
Figure 17: Testing evidence samples for validation, webhook, route guard, and order status checks	44

List of Abbreviations

Table 1: List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
COD	Cash on Delivery
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
DB	Database
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IDE	Integrated Development Environment
JS	JavaScript
JSON	JavaScript Object Notation
MySQL	My Structured Query Language
PDO	PHP Data Objects
PHP	Hypertext Preprocessor
REST	Representational State Transfer
RTDB	Realtime Database
SQL	Structured Query Language
UI	User Interface

Chapter 1

Introduction

Chapter 1

Introduction

Online shopping has become one of the most popular ways to buy products. Customers want to browse products, read specifications, compare prices, and pay safely from home. Electronic devices are especially popular online because customers need detailed information before they buy [1].

Small and local electronics stores in Palestine often do not have a proper online system. They use social media pages or phone calls to receive orders. This is slow, unorganized, and does not give customers a good experience.

This project builds a complete web-based electronic store for the Palestinian market. Customers can browse products by category, search for specific items, save favorites, manage a shopping cart, and pay using four different payment methods. The system keeps a full history of each customer's orders.

1.1 Problem Statement

Customers in Palestine who want to buy electronic devices from local stores face many problems. There is no easy way to see all products in one place, check stock, read specifications, or pay online. Most local stores use manual methods, which are slow, not organized, and difficult to track.

This project solves these problems by giving customers a proper online shopping experience with multiple payment options and order tracking, while giving store owners a clearer way to organize purchases.

1.2 Objectives

1.2.1 Main Objective

Build a complete web-based electronic store using Vue 3 for the frontend, plain PHP for the backend API, Firebase Realtime Database for product and user data, MySQL for order data, and Stripe for online payment.

1.2.2 Specific Objectives

- Define all system requirements for both customers and the store.
- Design a clear database structure using Firebase and MySQL.
- Build customer registration and login using Firebase Realtime Database.
- Display products from a product catalog with categories, search, and filters.
- Build a shopping cart with quantity control and favorites list.
- Build a checkout flow that supports four payment methods.
- Integrate Stripe for secure online payment with webhook confirmation.
- Build an order history page so customers can track their purchases.
- Test the system to make sure all features work correctly.

1.3 Scope and Limitations

Scope

- Web-based store for electronic devices: smartphones, laptops, tablets, accessories, wearables, TVs, and gaming products.
- Customer registration and login.
- Product browsing with category, brand, and price filters.
- Product detail pages with specifications.
- Shopping cart and favorites list.
- Checkout with four payment methods: Cash on Delivery, Credit Card, Bank Transfer, and Stripe.
- Order history page showing past purchases.
- Stripe payment with webhook confirmation.
- Responsive design for desktop, tablet, and mobile.

Limitations

- The system needs an internet connection to work.
- Stripe payment depends on Stripe being available.
- First version supports English only.
- No delivery tracking or shipping cost calculation.

1.4 Importance of the Project

This project is useful for customers because it gives them a clear and organized way to buy electronic devices online with multiple payment options. From an academic point of view, the project shows how to build a complete full-stack web application that connects a JavaScript frontend, a PHP REST API backend, two different databases, and a payment gateway.

1.5 Methodology

The team used the Agile Scrum method [10]. Scrum was chosen because it breaks the work into small parts called sprints. Each sprint produces working software that can be tested. This helped the team find and fix problems early and keep the project on schedule.

The five phases used in this project were:

1. Requirements — Listing all features needed by customers.
2. Design — Designing the database structure and user interface.
3. Development — Building the PHP backend and Vue.js frontend.
4. Testing — Testing the system after each sprint.
5. Deployment — Publishing the frontend on Netlify and the backend on Hostinger.

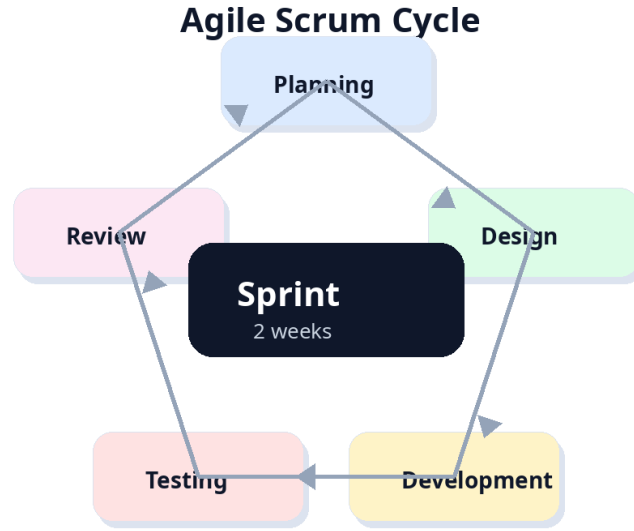


Figure 1: Agile Scrum model (research team design based on Scrum lifecycle [10])

1.6 Tools, Equipment, and Methods

Several tools and technologies were used to build this project. The selection was based on the requirements of the system, the need for security, ease of development, scalability, and availability of official documentation [2]-[12], [16]-[23].

Table 2: Tools and Technologies Used in the Project

Category	Tool / Technology	Purpose
Frontend Framework	Vue 3.2.13	Builds the user interface with reusable components and reactive data. [2]
State Management	Pinia 3.0.3	Manages shared state across Vue components (cart, auth, products, purchases). [3]
Routing	Vue Router 4.0.3	Handles navigation between pages and route guards for protected pages. [11]
CSS Framework	Bootstrap 5.3.8	Makes the design responsive and fast to build. [8]
Icons	Bootstrap Icons 1.13.1	Icon library used across the interface. [16]
Product & User Database	Firebase Realtime Database	Stores product catalog, user accounts, orders, purchases, and messages. [6]
Order Database	MySQL	Stores Stripe order records and payment status.
Firebase SDK	firebase 12.3.0	Connects Vue.js directly to Firebase Realtime Database for reads and writes. [6]
Backend Language	PHP 8.x (Vanilla / Plain PHP)	Handles Stripe payment API, Firebase product validation, and MySQL order management. [4]
PHP Data Access	PDO	Connects PHP to MySQL with prepared statements. [4]
Payment Gateway	Stripe API + PHP SDK	Processes online payments and sends webhook events to confirm payment. [5]
Local Server	XAMPP	Runs PHP and MySQL locally during development. [17]
Version Control	Git and GitHub	Manages code and team collaboration. [18]
IDE	Visual Studio Code	Main code editor. [19]

API Testing	Postman	Tests PHP endpoints and checks request and response data. [20]
Webhook Testing	Stripe CLI	Forwards Stripe webhook events to the local server during development. [5]
Frontend Hosting	Netlify	Hosts the Vue.js production build. [12]
Backend Hosting	Hostinger	Hosts the PHP API files. [21]
Package Managers	Composer and NPM	Manages PHP and JavaScript dependencies. [22], [23]

1.7 Timetable

Table 3: Agile Scrum Sprint Timeline

Sprint	Dates	Main Deliverables
Sprint 1	Oct 1–14, 2025	Project setup, requirements, technology choices, database design.
Sprint 2	Oct 15–28, 2025	User registration and login using Firebase, auth store, route guards.
Sprint 3	Oct 29–Nov 11, 2025	Product catalog, categories, filters, product detail pages, favorites.
Sprint 4	Nov 12–25, 2025	Shopping cart, checkout form, COD and bank transfer payment flow.
Sprint 5	Nov 26–Dec 9, 2025	Stripe payment integration, webhook, order status, cancel flow.
Sprint 6	Dec 10–23, 2025	My Purchases page, contact form, testing, bug fixes, deployment.
Sprint 7	Dec 24, 2025–Jan 20, 2026	Report writing, documentation review, final formatting, references, and preparation for discussion.

Table 4: Team Roles and Responsibilities

Team Member	Role	Responsibilities
Abdulrahman Ahmed Al-Dimassi	UI/UX Designer	Interface design, responsive screens, product cards, cart and checkout UI, and contribution to report writing.
Momin Ahmed Alwawi	Frontend Lead / Scrum Master	Vue.js components, Pinia stores, routing, testing, documentation, Firebase integration, sprint coordination, and report writing.
Abdulrahman Alaa Al-Jadili	Backend Lead	PHP API development, MySQL integration, Stripe payment, webhook implementation, and contribution to report writing.

Chapter 2

Related Works

Chapter 2

Related Works

This chapter looks at existing e-commerce platforms and technologies. The goal is to understand what features they have and how this project is different.

2.1 International E-commerce Platforms

2.1.1 Amazon [13]

Amazon is one of the largest online stores in the world. It has product search, categories, a shopping cart, customer reviews, order tracking, and secure payment. This project uses Amazon as a general model for the shopping workflow, especially how products are organized by category and how customers move from browsing to checkout [13].

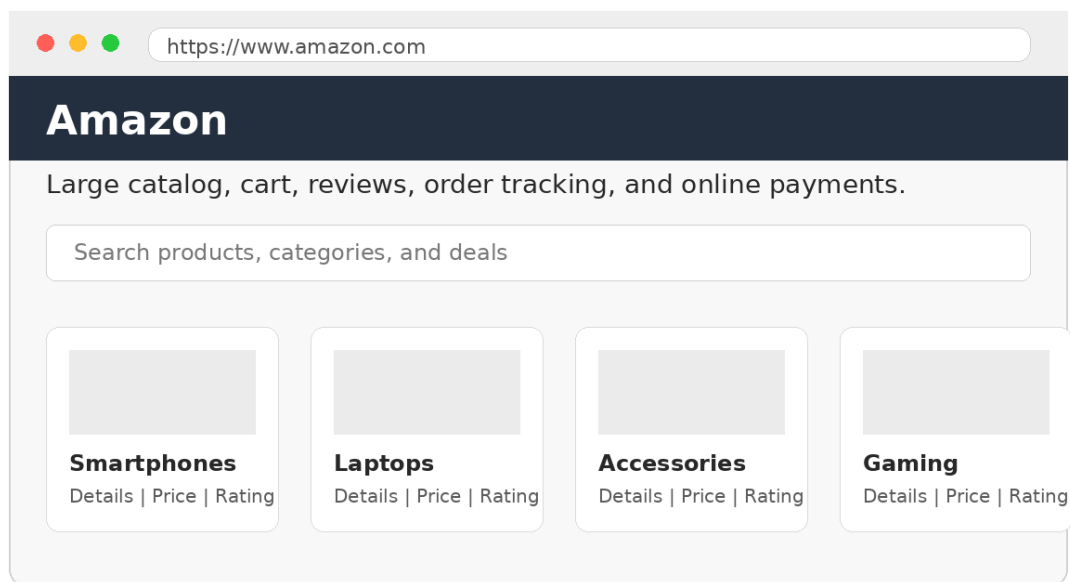


Figure 2: Amazon platform overview used as a reference for product browsing and checkout flow.

2.1.2 Newegg [14]

Newegg focuses on electronics and computer hardware. It shows detailed product specifications and supports advanced filtering. Because this project also sells electronic devices, Newegg was used as a reference for displaying product details, specifications, and technical data in a clear way [14].

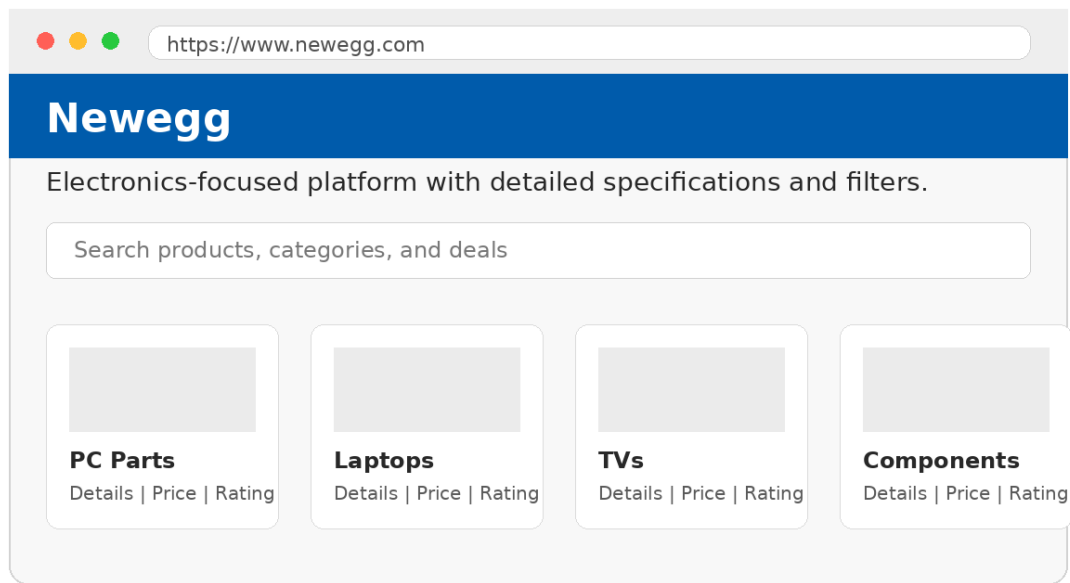


Figure 3: Newegg platform overview used as a reference for specifications and filters.

2.1.3 AliExpress [15]

AliExpress is a large international marketplace. The team used the idea of category browsing and order history from AliExpress. However, this project is a single store, not a marketplace with multiple sellers [15].

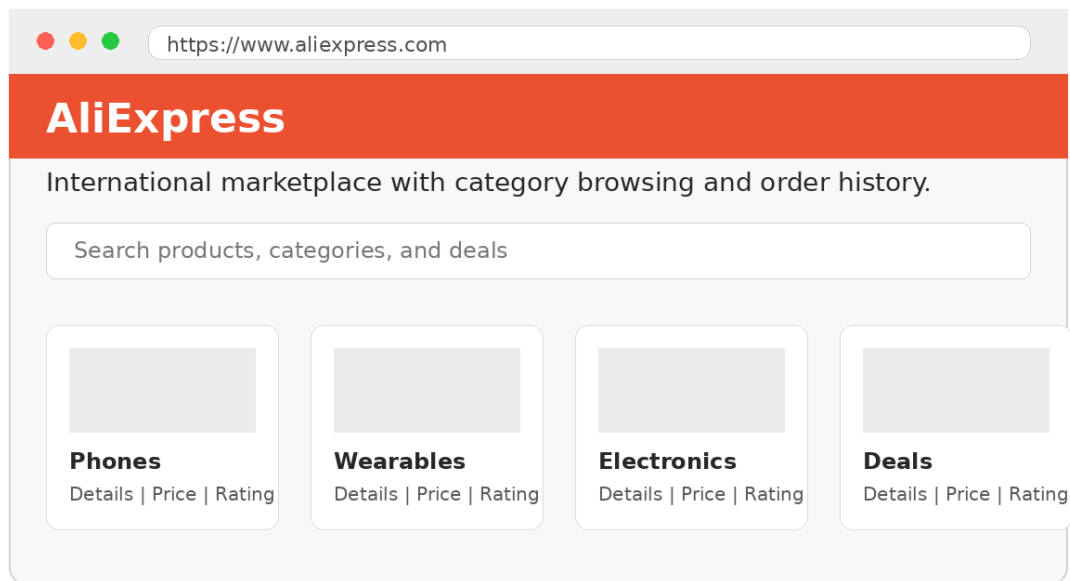


Figure 4: AliExpress platform overview used as a reference for category browsing and order history.

2.2 Technical Frameworks and Similar Systems

Vue 3 and Pinia

Vue 3 is a JavaScript framework for building user interfaces. It uses a component-based structure which makes it easy to build reusable product cards, cart items, and checkout forms. Pinia is the official state management library for Vue 3. It was used to manage the cart, favorites, authentication state, and purchase history across all pages. [2], [3].

Firebase Realtime Database

Firebase Realtime Database is a cloud database from Google. It stores and syncs data in real time. This project uses Firebase to store product data, user accounts, buyer information, sales records, purchase history, and contact messages. Vue.js connects to Firebase directly using the Firebase SDK. PHP also connects to Firebase using simple HTTP REST requests. [6].

Plain PHP REST API

Plain PHP means writing PHP without any framework like Laravel. Each PHP file handles one specific job. This makes the code easy to read and understand. The backend has six PHP files — each one is a separate REST API endpoint. [4].

Stripe Payment

Stripe provides APIs for online payments. This project uses Stripe Checkout, which creates a hosted payment page managed by Stripe. After payment, Stripe sends a webhook event to the PHP backend to confirm the result. This is reliable because it works even if the customer closes the browser. [5].

2.3 Comparison with Related Works

Table 5: Comparison of Related Works by Evaluation Criteria

Evaluation Criteria	Related Platforms	TechVerse Project
Technology used	Amazon, Newegg, and AliExpress use large-scale proprietary e-commerce infrastructures.	Custom academic implementation using Vue 3, PHP, Firebase, MySQL, and Stripe.
Platform type	Amazon and AliExpress are marketplaces; Newegg is an electronics-focused online store.	Single-store web-based electronic devices platform.
Speed and performance	Large platforms use advanced cloud infrastructure and optimized search engines.	Lightweight product browsing and filters using Vue/Pinia for fast frontend response.
Target audience	International users with broad product categories.	Local customers in Palestine looking for electronic devices from one organized store.
Payment methods	Online card/payment methods depending on country and platform rules.	Cash on Delivery, Credit Card, Bank Transfer, and Stripe Checkout.
Shipping methods	Advanced shipping networks, tracking, and international delivery.	Basic checkout and order history; delivery tracking is planned for future work.
Customization	Ready customer-facing workflows but not built for this academic project scope.	Custom-built to demonstrate full-stack development and integration skills.

2.4 The Difference Between Our System and Previous Systems

- The system is built for one specific store that sells electronic devices.
- It combines a Vue 3 frontend, a plain PHP REST API, Firebase Realtime Database, MySQL, and Stripe in one project.
- Firebase is used for product and user data. MySQL is used only for Stripe order records.

- The system supports four different payment methods in one checkout flow.
- A webhook system confirms Stripe payments reliably, even if the user closes the browser.
- Cart and favorites data persists across page refreshes using localStorage.

Chapter 3

Requirement Analysis

Chapter 3

Requirement Analysis

This chapter explains what the system should do. It describes the actors, use cases, functional requirements, non-functional requirements, and the database structure.

3.1 Requirement Collection

The team studied how e-commerce platforms work and what customers need when shopping online. The team also reviewed related platforms and the official documentation for Firebase, Vue 3, Pinia, and Stripe. This helped define the main features and choose the right tools [2], [3], [5], [6].

3.2 Actors

Guest

- Browse the home page, products page, and product detail pages.
- Search and filter products.
- Register for a new account.

Customer (Registered User)

- Log in and log out.
- Add products to the cart and favorites.
- Manage cart quantities.
- Go to checkout with shipping information.
- Pay using COD, Credit Card, Bank Transfer, or Stripe.
- View order history in My Purchases.
- Cancel pending Stripe orders.
- Send a message via the contact form.

Stripe (External Service)

Stripe receives the payment request, processes the card payment, and sends a webhook event back to the PHP backend to confirm the result.

3.3 Functional Requirements

Table 6: Functional Requirements

ID	Requirement
FR01	The system allows users to register by saving their name, email, and password to Firebase.
FR02	The system allows users to log in by checking their email and password against Firebase data.
FR03	The system displays products from the product catalog with category, brand, and price filters.
FR04	The system allows customers to view product details including specifications, price, and stock status.
FR05	The system allows customers to add products to the cart and update quantities.
FR06	The system allows customers to save products to a favorites list.
FR07	The system allows customers to checkout with shipping information and choose a payment method.
FR08	For COD, Card, and Bank Transfer: the system saves order data to Firebase (Buyers, Sales, My Purchases).

FR09	For Stripe: the system creates a Checkout Session through the PHP backend. Prices come from Firebase, not the frontend.
FR10	The system confirms Stripe payment through a webhook. Order status updates to paid in MySQL.
FR11	The system allows customers to cancel a pending Stripe order.
FR12	The system shows customers their order history from Firebase in My Purchases.
FR13	The system allows customers to send messages through the contact form saved to Firebase.
FR14	The system preserves cart and favorites data in localStorage across page refreshes.

3.4 Non-functional Requirements

Table 7: Non-functional Requirements

Quality	Description
Performance	Pages should load fast. Product filters and search should respond immediately since data is in the Pinia store.
Usability	Interfaces should be clean and simple. Buttons, forms, and product cards should be easy to use.
Security	Card data is handled only by Stripe — never stored on our server. PHP validates all inputs. Stripe webhook signature is verified before processing.
Reliability	The webhook system ensures payment confirmation even if the user closes the browser.
Availability	Frontend on Netlify and backend on Hostinger provide reliable uptime.
Scalability	Firebase and MySQL can support more data. The product catalog can be moved to Firebase in the future.
Maintainability	The code is split into separate Vue components, Pinia stores, and PHP files. Each part handles one specific job.
Compatibility	The interface works on modern browsers and different screen sizes using Bootstrap.

3.5 Use Case List

Table 8: Use Case List

ID	Name	Description
UC01	Register	User creates an account by entering name, email, and password.
UC02	Login	User logs in with email and password checked against Firebase.
UC03	Browse Products	Customer views the products page with search and filter options.
UC04	View Product Details	Customer opens a product page to see all details and specifications.
UC05	Manage Cart	Customer adds, removes, and updates quantities of cart items.
UC06	Manage Favorites	Customer saves or removes products from the favorites list.
UC07	Checkout (COD/Card/Bank)	Customer fills shipping info, selects non-Stripe payment, order saved to Firebase.
UC08	Checkout (Stripe)	Customer pays online through Stripe Checkout. Order saved in MySQL.
UC09	Cancel Stripe Order	Customer cancels a pending Stripe order. Status updated to cancelled.
UC10	View My Purchases	Customer views order history loaded from Firebase.
UC11	Contact Store	Customer sends a message through the contact form saved to Firebase.

3.6 Use Case Descriptions

Table 9: Registration Use Case

Field	Description
Actor	Guest
Pre-condition	User is not logged in.
Steps	1. Open signup page. 2. Enter name, email, password. 3. System checks Firebase if email exists. 4. If not, save user to Firebase /Sign up. 5. Redirect to login after 2 seconds.
Post-condition	User account saved in Firebase. User can now log in.
Alternative	If email already exists, show error message.

Table 10: Stripe Use Case

Field	Description
Actors	Customer and Stripe
Pre-condition	Customer is logged in and has products in the cart.
Steps	1. Customer fills shipping info on checkout page. 2. Selects Stripe payment method. 3. Clicks Pay with Stripe. 4. Vue sends cart (product IDs and quantities only) to PHP. 5. PHP fetches real prices from Firebase. 6. PHP creates MySQL order with pending status. 7. PHP creates Stripe session and returns checkout URL. 8. Browser redirects to Stripe hosted page. 9. Customer enters card details on Stripe. 10. Stripe sends webhook to PHP. 11. PHP updates MySQL order to paid. 12. Stripe redirects to success page. 13. Cart is cleared.
Post-condition	Order saved in MySQL with status paid. Cart cleared.
Alternative	If customer cancels on Stripe, redirected to cancel page. PHP updates order to cancelled.

3.7 Database Structure

The project uses two databases. Firebase Realtime Database stores product data, user accounts, buyer info, sales, purchase history, and contact messages. MySQL stores only Stripe order records and payment status. Vue.js connects to Firebase directly using the Firebase SDK. PHP connects to both Firebase (via HTTP REST) and MySQL (via PDO).

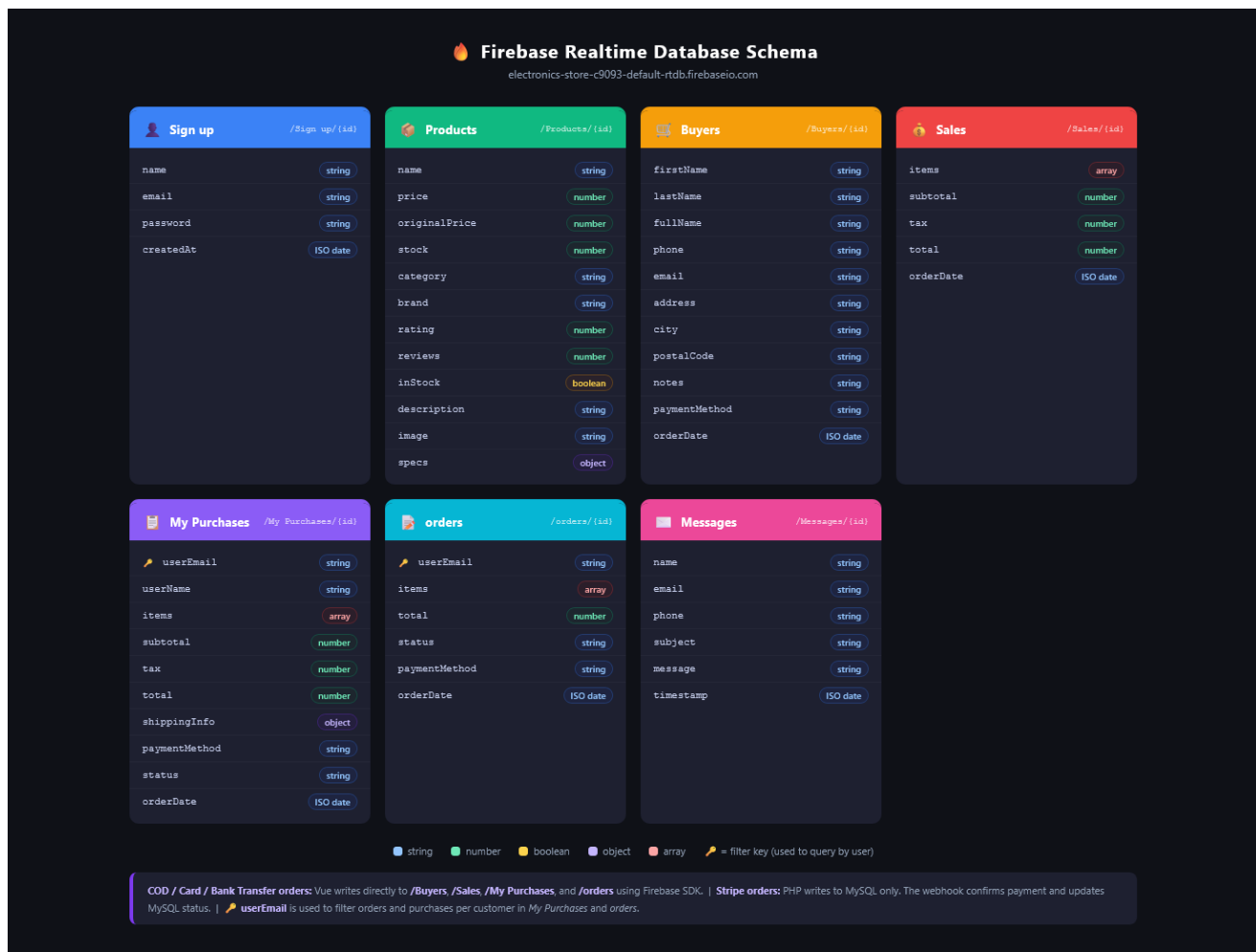


Figure 5: Firebase Realtime Database structure.

3.8 Database Tables Description

Table 11: Firebase Realtime Database Schema

Node	Field	Type	Description
/Sign up	name	string	User's full name
/Sign up	email	string	User's email address (used for login)
/Sign up	password	string	User's password
/Sign up	createdAt	string	ISO date when account was created
/Products/{id}	name	string	Product name e.g. iPhone 15 Pro Max
/Products/{id}	price	number	Current price in USD
/Products/{id}	originalPrice	number	Original price before discount
/Products/{id}	stock	number	Available quantity
/Products/{id}	image	string	Image path
/Products/{id}	category	string	e.g. smartphones, laptops, tablets
/Products/{id}	brand	string	e.g. Apple, Samsung
/Products/{id}	rating	number	Average rating out of 5
/Products/{id}	reviews	number	Number of reviews

/Products/{id}	inStock	boolean	Whether product is available
/Products/{id}	description	string	Product description
/Products/{id}	specs	object	Technical specs: screen, processor, RAM, storage, camera, battery
/Buyers	firstName	string	Buyer's first name from checkout form
/Buyers	lastName	string	Buyer's last name
/Buyers	phone	string	Phone number
/Buyers	email	string	Email
/Buyers	address	string	Shipping address
/Buyers	city	string	City
/Buyers	postalCode	string	Postal code
/Buyers	paymentMethod	string	COD, card, or bank
/Buyers	orderDate	string	ISO date of order
/Sales	items	array	Order items: [{id, name, price, quantity, total, image}]
/Sales	subtotal	number	Order subtotal
/Sales	tax	number	Tax amount
/Sales	total	number	Total including tax
/Sales	orderDate	string	ISO date
/My Purchases	userEmail	string	Email used to filter orders per customer
/My Purchases	userName	string	Customer name
/My Purchases	items	array	Purchased items list
/My Purchases	shippingInfo	object	Shipping address details
/My Purchases	paymentMethod	string	Payment method used
/My Purchases	status	string	Order status
/My Purchases	orderDate	string	ISO date
/Messages	name	string	Sender's name
/Messages	email	string	Sender's email
/Messages	phone	string	Sender's phone
/Messages	subject	string	Message subject
/Messages	message	string	Message content
/Messages	timestamp	string	ISO date sent

3.9 MySQL Schema (Orders Table — Stripe Only)

Table 12: MySQL Orders Table Schema

Column	Type	Constraints	Description
id	INT	AUTO_INCREMENT PRIMARY KEY	Unique order ID
firebase_uid	VARCHAR(128)	NOT NULL	Customer email or Firebase user ID
product_id	VARCHAR(100)	NOT NULL	Comma-separated product IDs
product_name	VARCHAR(255)	NOT NULL	Product names e.g. iPhone 15 x2
amount	INT	NOT NULL	Total in cents (99900 = \$999.00)
currency	VARCHAR(10)	DEFAULT 'usd'	Always usd

stripe_session_id	VARCHAR(255)	UNIQUE	Stripe session ID cs_test_...
status	ENUM	DEFAULT 'pending'	pending / paid / failed / cancelled
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Auto-set on insert
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Auto-updated on change

Chapter 4

Implementation

Chapter 4

Implementation

This chapter explains how the system was built. It describes the system architecture, the PHP backend, the Vue.js frontend, the Pinia stores, the database connections, and the payment flows.

4.1 System Architecture

The system has three layers working together:

- Frontend: Vue 3 running in the browser. Handles all user interface pages, product display, cart management, and checkout.
- Backend: Plain PHP files on XAMPP (local) or Hostinger (production). Each file is one REST API endpoint. Used only for Stripe payment processing.
- Data Layer: Firebase Realtime Database for products, users, orders, and messages. MySQL for Stripe order records. Stripe for payment processing.

System Architecture Diagram

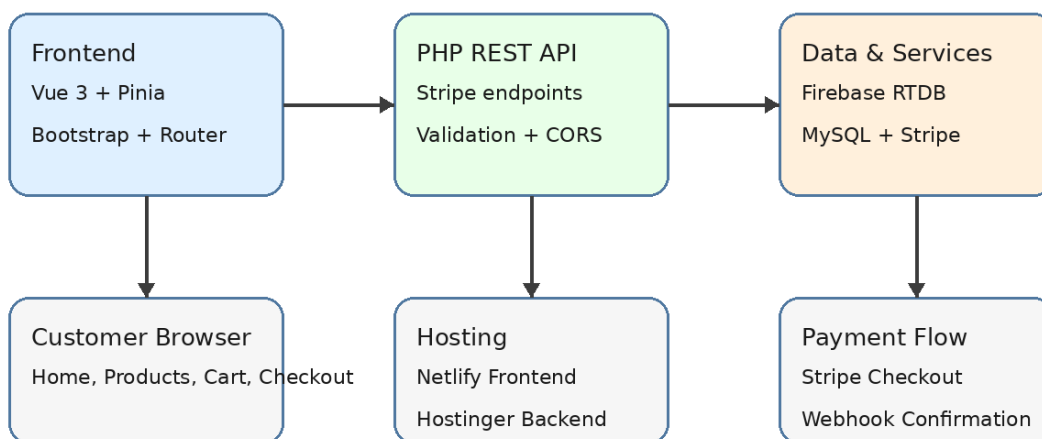


Figure 6: System Architecture Diagram.

4.2 PHP Backend — File Structure

Table 13: PHP Backend Endpoints

File	Method	Input	Output	Job
config.php	—	—	—	Loads .env variables, sets CORS headers, handles OPTIONS preflight.
db.php	—	—	PDO connection	Creates MySQL connection using PDO. Creates orders table if not exists.

firebase.php	—	productId (string)	{name, price, stock}	Fetches product data from Firebase via HTTP GET. Used by creatCheckout.php.
creatCheckout.php	POST	{cart_items, firebase_uid}	{checkout_url, order_id}	Validates cart, fetches prices from Firebase, saves pending order to MySQL, creates Stripe session.
webhook.php	POST	Stripe payload + signature	{received: true}	Verifies Stripe signature. Updates order to paid or failed in MySQL.
cancelOrder.php	POST	{order_id, firebase_uid}	{success: true}	Updates order status to cancelled in MySQL. Only works if status is pending.
orderStatus.php	GET	order_id, firebase_uid (query)	order object	Returns order details from MySQL to Vue.

4.3 config.php

This file is required at the top of every PHP file. It loads all credentials from the .env file and sets the CORS headers that allow Vue.js to send requests from a different origin.

```
<?php

require_once __DIR__ . '/vendor/autoload.php';

$dotenv = Dotenv\Dotenv::createImmutable(__DIR__);
$dotenv->load();

define('STRIPE_SECRET_KEY',    getenv('STRIPE_SECRET_KEY'));
define('STRIPE_WEBHOOK_SECRET', getenv('STRIPE_WEBHOOK_SECRET'));
define('FIREBASE_PROJECT_ID',  getenv('FIREBASE_PROJECT_ID'));
define('DB_HOST',    getenv('DB_HOST'));
define('DB_NAME',    getenv('DB_NAME'));
define('DB_USER',    getenv('DB_USER'));
define('DB_PASS',    getenv('DB_PASS'));
define('SUCCESS_URL', getenv('SUCCESS_URL'));
define('CANCEL_URL',  getenv('CANCEL_URL'));
```

```

header('Content-Type: application/json');

header('Access-Control-Allow-Origin: ' . getenv('FRONTEND_ORIGIN'));

header('Access-Control-Allow-Methods: POST, GET, OPTIONS');

header('Access-Control-Allow-Headers: Content-Type');


if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {

    http_response_code(200); exit;

}

```

Note: CORS headers are needed because Vue.js (Netlify) and PHP (Hostinger) are on different domains. Without these headers the browser blocks all requests

4.4 db.php — MySQL Connection

Creates a single PDO connection and reuses it. Uses static variable so the database connection is made only once per request.

```

function getDB(): PDO {

    static $pdo = null;

    if ($pdo === null) {

        $dsn = 'mysql:host='.DB_HOST.';dbname='.DB_NAME.';charset=utf8mb4';

        $pdo = new PDO($dsn, DB_USER, DB_PASS, [

            PDO::ATTR_ERRMODE                => PDO::ERRMODE_EXCEPTION,

            PDO::ATTR_DEFAULT_FETCH_MODE    => PDO::FETCH_ASSOC,

            PDO::ATTR_EMULATE_PREPARES     => false,

        ]);

    }

    return $pdo;

}

```

4.5 firebase.php — Reading Product Prices

Fetches product name, price, and stock from Firebase Realtime Database using a plain HTTP GET request. No SDK needed — Firebase has a built-in REST API.

```

function getProductFromFirebase(string $productId): ?array {

```

```

$url = "https://".FIREBASE_PROJECT_ID."-default-rtdb.firebaseio.com"
    . "/Products/" . urlencode($productId) . ".json";

$context = stream_context_create(['http'=>['method'=>'GET','timeout'=>10]]);
$response = @file_get_contents($url, false, $context);

if (!$response) return null;

$data = json_decode($response, true);
if (!$data || isset($data['error'])) return null;

return [
    'name' => $data['name'] ?? 'Unknown',
    'price' => $data['price'] ?? 0,
    'stock' => $data['stock'] ?? 0,
];
}

```

Note: Prices always come from Firebase on the server side. Vue.js only sends product IDs and quantities. This prevents customers from changing prices in the browser.

4.6 creatCheckout.php — Stripe Session

The main payment endpoint. Receives the cart from Vue, validates prices against Firebase, saves a pending order, creates a Stripe session, and returns the checkout URL.

```

// 1. Parse input
$input = json_decode(file_get_contents('php://input'), true);

$cart_items = $input['cart_items']; // [{product_id, quantity}]
$firebase_uid = $input['firebase_uid']; // customer email

// 2. Validate each item against Firebase
foreach ($cart_items as $item) {
    $product = getProductFromFirebase($item['product_id']);

    if (!$product) { respond 404; }
}

```

```
if ($product['stock'] < $item['quantity']) { respond 400; }

$line_items[] = [
    'price_data' => [
        'currency'      => 'usd',
        'unit_amount'   => $product['price'] * 100,
        'product_data' => ['name' => $product['name']],
    ],
    'quantity' => $item['quantity'],
];
}

// 3. Save pending order to MySQL
$stmt = getDB()->prepare(
    'INSERT INTO orders (firebase_uid,product_id,product_name,amount,currency)
    VALUES (?, ?, ?, ?, ?)'
);

$stmt->execute([$firebase_uid, $ids, $names, $total*100, 'usd']);
$order_id = getDB()->lastInsertId();

// 4. Create Stripe session
\Stripe\Stripe::setApiKey(STRIPE_SECRET_KEY);
$session = \Stripe\Checkout\Session::create([
    'line_items' => $line_items,
    'mode'       => 'payment',
    'success_url' => SUCCESS_URL . '?order_id=' . $order_id,
    'cancel_url'  => CANCEL_URL . '?order_id=' . $order_id,
    'metadata'    => ['order_id' => $order_id],
]);

// 5. Save session ID and return URL
getDB()->prepare('UPDATE orders SET stripe_session_id=? WHERE id=?')
```

```

->execute([$session->id, $order_id]);

echo json_encode(['checkout_url'=>$session->url, 'order_id'=>$order_id]);

```

4.7 webhook.php — Payment Confirmation

Stripe calls this endpoint after a payment is completed or a session expires. PHP verifies the Stripe signature before processing anything.

```

$payload = file_get_contents('php://input');
$signature = $_SERVER['HTTP_STRIPE_SIGNATURE'];

$event = \Stripe\Webhook::constructEvent(
    $payload, $signature, STRIPE_WEBHOOK_SECRET
);

if ($event->type === 'checkout.session.completed') {
    $order_id = $event->data->object->metadata->order_id;
    getDB()->prepare('UPDATE orders SET status="paid" WHERE id=?')
        ->execute([$order_id]);
}

if ($event->type === 'checkout.session.expired') {
    $order_id = $event->data->object->metadata->order_id;
    getDB()->prepare('UPDATE orders SET status="failed" WHERE id=?')
        ->execute([$order_id]);
}

http_response_code(200);
echo json_encode(['received' => true]);

```

Note: The webhook is server-to-server. Stripe calls it directly. This means the order is confirmed even if the customer closes their browser before reaching the success page

4.8 cancelOrder.php — Cancel an Order

When the customer clicks back on the Stripe page, Vue calls this endpoint to immediately update the order to cancelled.

```
$input      = json_decode(file_get_contents('php://input'), true);
$order_id   = (int) ($input['order_id'] ?? 0);
$firebase_uid = $input['firebase_uid'] ?? '';

$stmt = getDB()->prepare(
    'UPDATE orders SET status="cancelled"
    WHERE id=? AND firebase_uid=? AND status="pending"'
);
$stmt->execute([$order_id, $firebase_uid]);

if ($stmt->rowCount() === 0) {
    http_response_code(404);
    echo json_encode(['error' => 'Order not found or already processed']);
    exit;
}

echo json_encode(['success' => true]);
```

Note: The AND status='pending' check makes sure a paid order cannot be cancelled

4.9 Frontend — Vue.js Pages

Table 14: Vue.js Pages

Page	Path	What it Shows	Data Source
HomePage	/	Hero carousel, categories, featured products, store features.	productsStore
ProductsPage	/products	Full product grid with search, category, brand, price filters.	productsStore
ProductDetailPage	/product/:id	Product image, specs table, add to cart, add to favorites.	productsStore

CartPage	/cart	Cart items, quantity controls, order summary, proceed to checkout.	cartStore
CheckoutPage	/checkout	Shipping form, payment method selector, order summary with tax.	cartStore, authStore, purchasesStore
FavoritesPage	/favorites	Saved products grid.	cartStore (wishlist)
MyPurchasesPage	/my-purchases	Order history with items, shipping info, payment method, totals.	purchasesStore → Firebase
LoginPage	/login	Email and password login form.	authStore → Firebase
SignupPage	/signup	Name, email, password registration form.	Firebase directly
StripeSuccessPage	/stripe-success	Payment success message. Clears cart.	cartStore
StripeCancelPage	/stripe-cancel	Payment cancelled message. Calls cancelOrder.php.	PHP backend
AboutPage	/about	Company story, values, statistics.	Static
ContactPage	/contact	Contact form. Saves message to Firebase.	Firebase directly

4.10 Frontend — Pinia Stores

Table 15: Pinia Stores

Store	State	Key Actions
authStore	user, isAuthenticated	login (reads Firebase), logout, save/load from localStorage
productsStore	products (12 items), categories, brands, filters	setFilter, resetFilters, getProductById, filteredProducts, featuredProducts
cartStore	items [], wishlist []	addToCart, removeFromCart, updateQuantity, clearCart, addToWishlist, save/load from localStorage
purchasesStore	purchases []	addPurchase (POST to Firebase), loadPurchases (GET from Firebase), userPurchases filtered by email

Note: cartStore and authStore save their state to localStorage so data is not lost when the page is refreshed.

4.11 Frontend — Vue Router

Table 16: Vue Router Routes

Path	Component	Access
/	HomePage	All users
/about	AboutPage	All users
/products	ProductsPage	All users
/product/:id	ProductDetailPage	All users
/contact	ContactPage	All users
/cart	CartPage	Authenticated customers only
/favorites	FavoritesPage	Authenticated customers only
/checkout	CheckoutPage	Authenticated customers only

/my-purchases	MyPurchasesPage	Authenticated customers only
/stripe-success	StripeSuccessPage	All users
/stripe-cancel	StripeCancelPage	All users
/login	LoginPage	Guests only (redirect if already logged in)
/signup	SignupPage	Guests only (redirect if already logged in)

Note: Route guards use `authStore.isAuthenticated`. Protected routes redirect to `/login` if not authenticated

4.12 Authentication Flow

Registration

1. User opens `/signup` and fills name, email, password.
2. Vue checks Firebase `/Sign up` for existing email.
3. If email not found, Vue POSTs new user to Firebase `/Sign up`.
4. Success: redirect to `/login` after 2 seconds.

Login

1. User opens `/login` and enters email and password.
2. Vue fetches all users from Firebase `/Sign up`.
3. Finds matching email and password.
4. If found, `authStore.login()` saves user to state and `localStorage`.
5. User is redirected to home page.

4.13 Payment Flows

Flow 1 — COD, Credit Card, Bank Transfer

1. Customer fills shipping form and selects COD, card, or bank transfer.
2. Vue saves buyer data to Firebase `/Buyers`.
3. Vue saves sale data to Firebase `/Sales`.
4. Vue saves purchase to Firebase `/My Purchases` (linked by email).
5. `purchasesStore` records the purchase locally.
6. Cart is cleared. User redirected to home.

Note: These three payment methods do not involve PHP. Vue writes directly to Firebase.

Flow 2 — Stripe Online Payment

Step	Actor	Action
1	Customer	Selects Stripe and clicks Pay with Stripe.
2	Vue.js	Builds [{product_id, quantity}] — no prices sent.
3	Vue → PHP	POST /creatCheckout.php with cart and firebase_uid.
4	PHP → Firebase	GET real price and stock for each product.
5	PHP → MySQL	INSERT order with status=pending. Gets order_id.
6	PHP → Stripe	Create Checkout Session with real line items.
7	PHP → MySQL	UPDATE order with stripe_session_id.
8	PHP → Vue	Return {checkout_url, order_id}.
9	Browser	Redirect to Stripe hosted checkout page.
10	Customer	Enters card details on Stripe's page.
11	Stripe → PHP	POST webhook: checkout.session.completed.
12	PHP → MySQL	UPDATE status = paid.
13	Stripe → Browser	Redirect to /stripe-success?order_id=X.
14	Vue	Clear cart. Show success message.

Flow 3 — Stripe Cancellation

1. Customer clicks back on the Stripe page.
2. Stripe redirects to /stripe-cancel?order_id=X.
3. StripeCancelPage calls POST /cancelOrder.php.
4. PHP updates MySQL order status to cancelled.
5. Customer sees cancellation message. Cart items preserved.

Note: All Stripe payment functionality demonstrated in this project was tested using Stripe's Sandbox (Test Mode) environment with test API keys and test payment cards. This allowed the complete payment workflow, including Checkout Sessions and webhook confirmation, to be validated safely without involving real financial transactions.

4.14 Security

- Card details are handled only by Stripe — they never reach our server.
- Stripe webhook signature is verified using STRIPE_WEBHOOK_SECRET before any database update.
- Product prices come from Firebase on the server side — customers cannot change prices.
- PHP validates all input before using it in queries.

The following simplified validation example shows how the backend rejects invalid checkout requests before creating any order:

```
$input = json_decode(file_get_contents('php://input'), true);
if (empty($input['cart_items']) || empty($input['firebase_uid'])) {
    http_response_code(400);
    echo json_encode(['error' => 'Invalid checkout request']);
    exit;
}
```

- PDO prepared statements protect against SQL injection.
- cancelOrder.php checks both order_id and firebase_uid so customers can only cancel their own orders.
- Route guards in Vue Router prevent guests from accessing checkout, cart, and order history pages.
- HTTPS is used in production for both Netlify and Hostinger.

4.15 Deployment

The Vue.js frontend is deployed on Netlify. Because Vue uses client-side routing, a _redirects file was added to the public folder:

```
/*      /index.html      200
```

This tells Netlify to serve index.html for all routes and let Vue Router handle navigation. Without this file, direct URLs like /stripe-success return a 404 error.

The PHP backend is deployed on Hostinger. All credentials are stored in the .env file. CORS headers in config.php allow requests from the Netlify domain. Stripe webhook URL is registered in the Stripe Dashboard pointing to the Hostinger server.

Published platform URL: <https://electronics-store-project.netlify.app>



QR Code: Published platform link for quick access.

Chapter 5

User Interface

5.1 User Interface

The store uses Bootstrap 5 and Bootstrap Icons. All pages are responsive and work on desktop, tablet, and mobile screens.

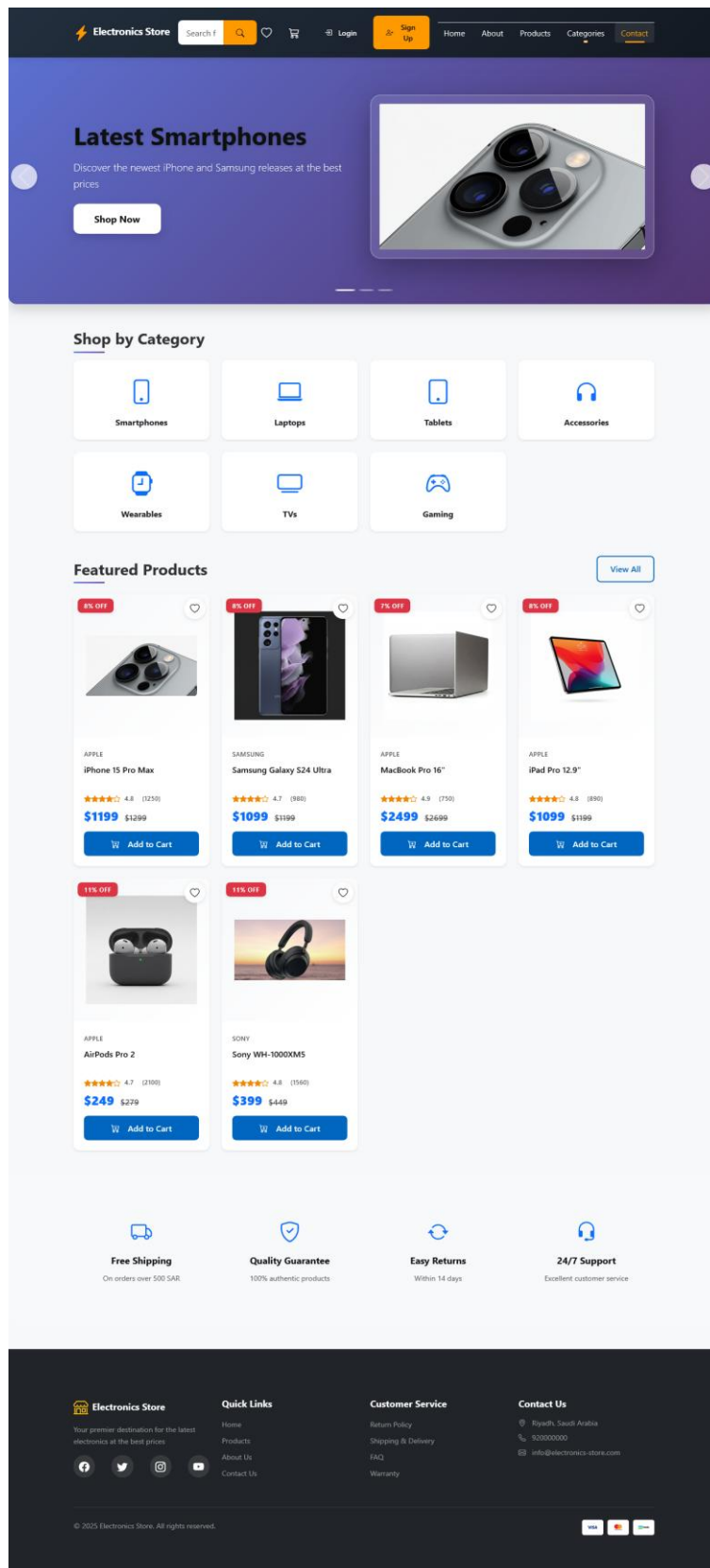


Figure 7: Hero carousel with categories and featured products.

This home interface presents the platform identity, navigation header, featured products, and main categories. It helps customers start browsing quickly and access product sections, login, cart, and other main pages.

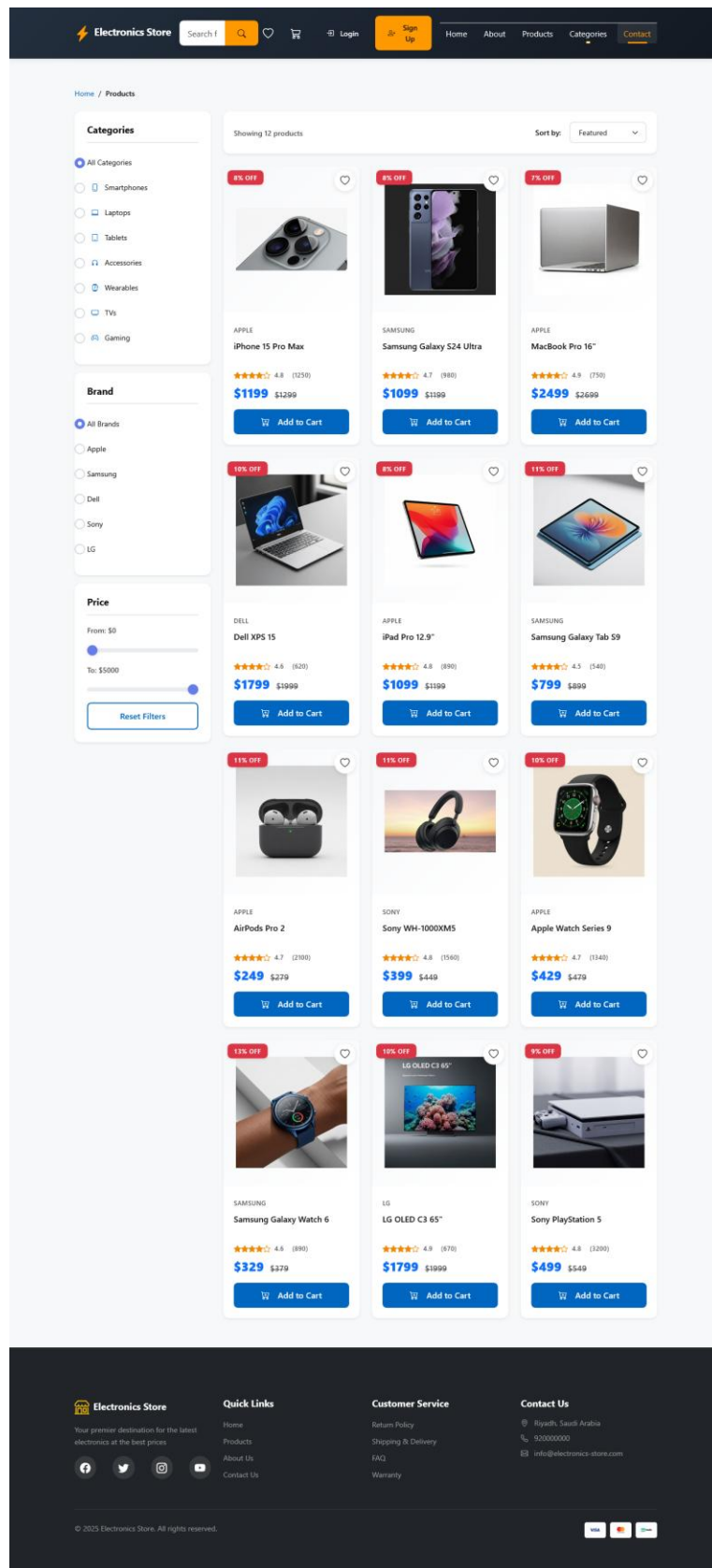


Figure 8: Product grid with category, brand, and price filters.

This products interface displays the available electronic devices in a clear grid. Customers can search, filter by category or brand, and narrow results by price to find suitable products faster.

[Home](#)
[About](#)
[Products](#)
[Categories](#)
[Contact](#)

[Home](#) / [Products](#) / Dell XPS 15

Dell XPS 15

★★★★☆ 4.6 (620 reviews)

\$1799 ~~\$1999~~ Save \$200

In Stock

Description

Powerful laptop for professionals with InfinityEdge display

Quantity:

+

-

Free shipping on orders over \$500

Free returns within 14 days

1 year warranty

Technical Specifications

Screen	15.6-inch 4K OLED	Processor	Intel Core i9-13900H
RAM	32GB DDR5	Storage	1TB SSD
Graphics Card	NVIDIA RTX 4060	Battery	Up to 12 hours

Related Products

7% OFF

APPLE

MacBook Pro 16"

★★★★☆ 4.9 (750)

\$2499 ~~\$2699~~

Electronics Store

Your premier destination for the latest electronics at the best prices

Quick Links

- Home
- Products
- About Us
- Contact Us

Customer Service

- Return Policy
- Shipping & Delivery
- FAQ
- Warranty

Contact Us

- Riyadh, Saudi Arabia
- 920000000
- info@electronics-store.com

© 2025 Electronics Store. All rights reserved.

Figure 9: Product details with specifications table, add to cart, and favorites.

This page shows full product information such as image, price, stock status, and technical specifications. It also allows the customer to add the item to the cart or save it to the favorites list.

The screenshot displays the login interface of the 'Electronics Store'. At the top, a dark navigation bar contains the store's logo, a search bar, and links for Home, About, Products, Categories, and Contact. A central white login box features the title 'Login' and two input fields for 'Email' and 'Password'. Below these fields is a blue 'Login' button and a link to 'Create a new account'. The footer section includes social media icons, a 'Quick Links' menu, a 'Customer Service' menu, and 'Contact Us' information, all set against a dark background.

Electronics Store

Search f

Home About Products Categories Contact

Login

Email

Enter your email

Password

Enter your password

Login

Don't have an account? [Create a new account](#)

Electronics Store

Your premier destination for the latest electronics at the best prices

Quick Links

- Home
- Products
- About Us
- Contact Us

Customer Service

- Return Policy
- Shipping & Delivery
- FAQ
- Warranty

Contact Us

- Riyadh, Saudi Arabia
- 920000000
- info@electronics-store.com

© 2025 Electronics Store. All rights reserved.

VISA Mastercard

Figure 10: Login form with email and password.

This interface allows registered users to access their accounts using email and password. After successful login, the system stores the user state and allows access to protected pages.

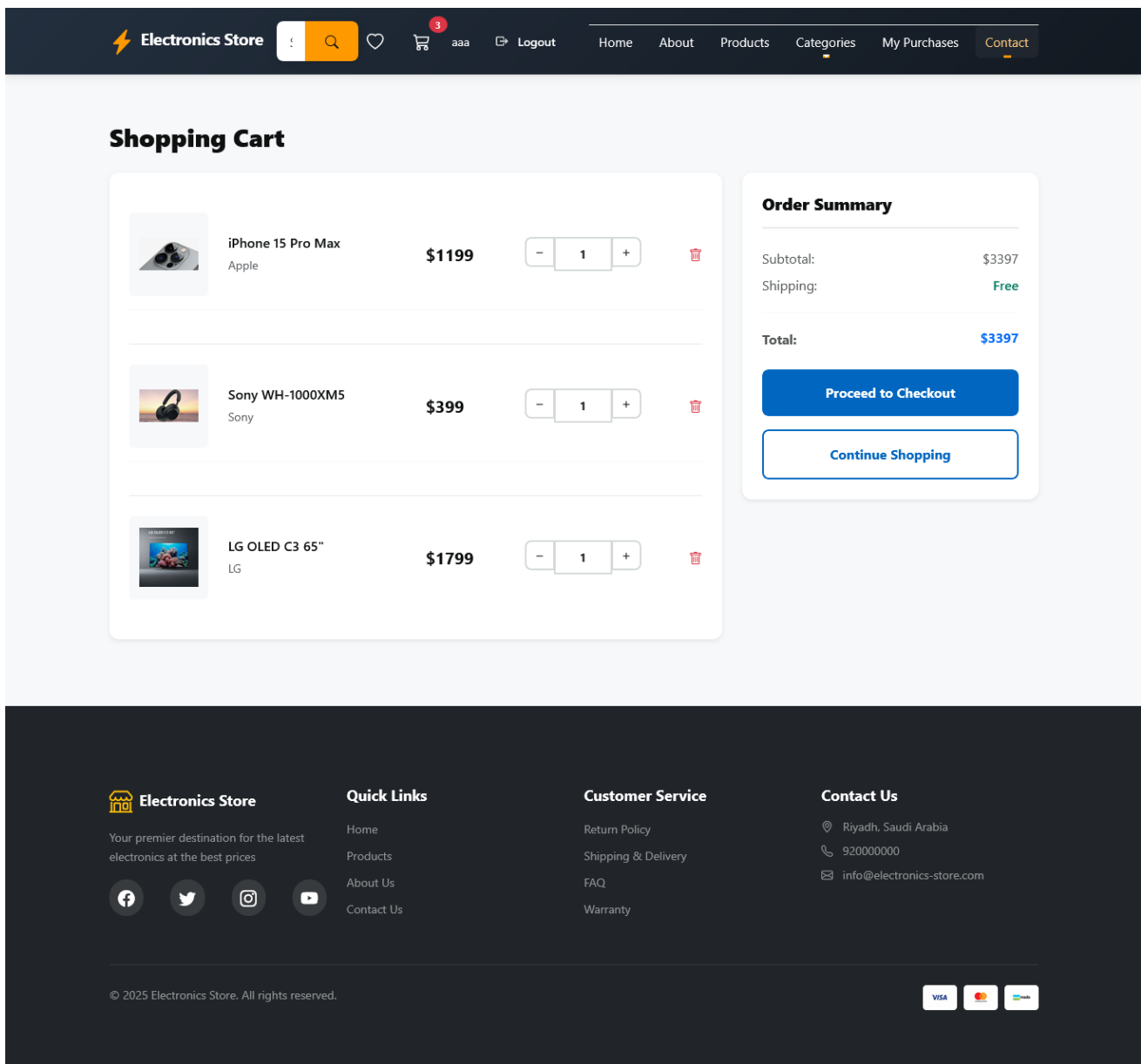


Figure 11: Cart with item list, quantity controls, and order summary.

The cart page lets customers review selected products, change quantities, remove items, and view subtotal, tax, and final total before moving to checkout.





aaa

[Home](#)
[About](#)
[Products](#)
[Categories](#)
[My Purchases](#)

Checkout

Shipping Information

First Name *

Last Name *

Phone Number *

Email *

Address *

City *

Postal Code *

Order Notes (Optional)

Order Summary

iPhone 15 Pro Max x 1	\$1199
Sony WH-1000XM5 x 1	\$399
LG OLED C3 65" x 1	\$1799
Subtotal:	\$3397
Shipping:	Free
Tax (15%):	\$510
Total:	\$3907

 Secure and encrypted transaction

Payment Method

☒  Cash on Delivery

☐  Credit / Debit Card

☐  Bank Transfer

☐  Pay securely with Stripe



Electronics Store

Your premier destination for the latest electronics at the best prices






Quick Links

- [Home](#)
- [Products](#)
- [About Us](#)
- [Contact Us](#)

Customer Service

- [Return Policy](#)
- [Shipping & Delivery](#)
- [FAQ](#)
- [Warranty](#)

Contact Us

-  Riyadh, Saudi Arabia
-  920000000
-  info@electronics-store.com

© 2025 Electronics Store. All rights reserved.





Figure 12: Checkout form with shipping info and payment method selection.

The checkout page collects shipping data and payment choice. It supports Cash on Delivery, Credit Card, Bank Transfer, and Stripe, then saves the order according to the selected payment flow.

The image shows a Stripe hosted checkout page. On the left, there's a shopping cart titled 'New business sandbox' with a 'Sandbox' button. The cart contains three items: 'iPhone 15 Pro Max' priced at ₪3,621.40, 'Sony WH-1000XM5' at ₪1,205.12, and 'LG OLED C3 65"' at ₪5,433.61. Above the items, there's a 'Choose a currency:' section with two options: '₪10,260.14' (selected) and '\$3,397.00'. Below this, it says '1 USD = 3.0204 ILS'. On the right, the 'Contact information' section has an 'Email' field with 'email@example.com'. Below that is the 'Payment method' section, which is currently set to 'Card'. The 'Card information' sub-section includes a card number field (1234 1234 1234 1234), a 'MM / YY' field, a 'CVC' field, and a dropdown for 'Country or region' (currently set to 'Palestinian Territories'). There's also a checkbox for 'Save my information for faster checkout' with a note about security. At the bottom, there's a large blue 'Pay' button. Below the button, it says 'Powered by stripe' with links to 'Terms' and 'Privacy'.

New business sandbox **Sandbox**

Choose a currency:

₪10,260.14 \$3,397.00

1 USD = 3.0204 ILS

iPhone 15 Pro Max	₪3,621.40
Sony WH-1000XM5	₪1,205.12
LG OLED C3 65"	₪5,433.61

Contact information

Email

email@example.com

Payment method

Card

Card information

1234 1234 1234 1234

MM / YY CVC

Cardholder name

Full name on card

Country or region

Palestinian Territories

☐ Save my information for faster checkout

Pay securely at New business sandbox and everywhere [Link](#) is accepted.

Pay

Powered by stripe | Terms Privacy

Figure 13: Stripe hosted checkout page.

This secure payment interface is managed by Stripe. Customers enter card details on Stripe's hosted page, while sensitive card data never reaches the project server.

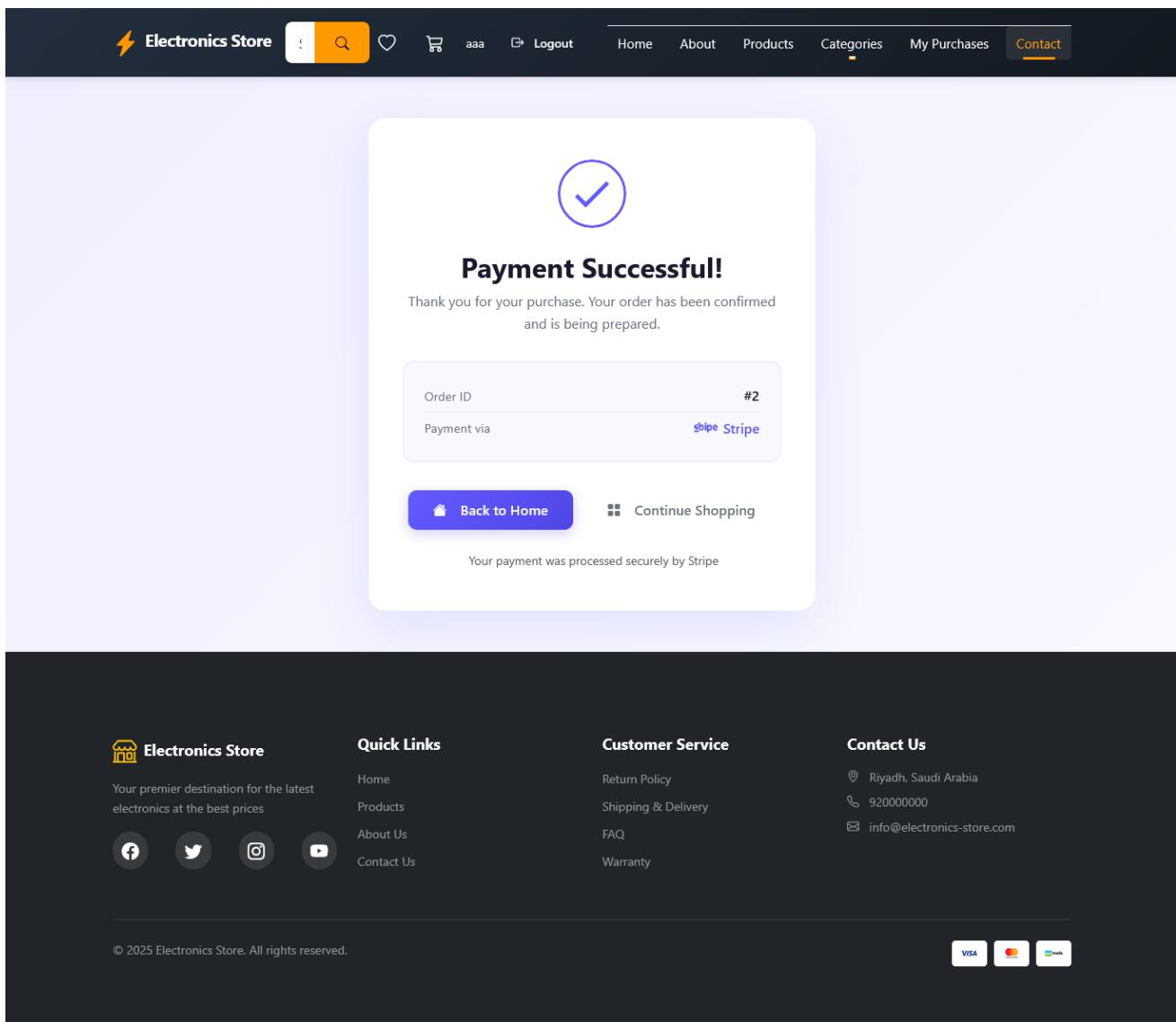


Figure 14: Success page after Stripe payment confirmed.

The success page informs the customer that the online payment has been completed. It appears after Stripe redirects the customer back to the platform and the order status is confirmed.


Electronics Store





aaa


Logout

Home

About

Products

Categories

My Purchases

Contact

My Purchases

Order #-OmiVzvr6hrvE4BBXbeV
March 2, 2026 at 02:26 PM

Processing

Items:



Samsung Galaxy S24 Ultra
Quantity: 1 × \$1099

\$1099



MacBook Pro 16"
Quantity: 1 × \$2499

\$2499

Order Summary

Subtotal: \$3598
Tax: \$540
Shipping: Free
Total: \$4138

Shipping Information:

Name: aa aa
Address: ad, qdad, 1111
Phone: 0599656764
Payment: stripe


Electronics Store
Your premier destination for the latest electronics at the best prices

Quick Links

Home
Products
About Us
Contact Us

Customer Service

Return Policy
Shipping & Delivery
FAQ
Warranty

Contact Us

Riyadh, Saudi Arabia
920000000
info@electronics-store.com

© 2025 Electronics Store. All rights reserved.





Figure 15: Order history with items, shipping info, and payment details.

The My Purchases page shows each customer their previous orders, purchased items, shipping information, payment method, status, and total amount.

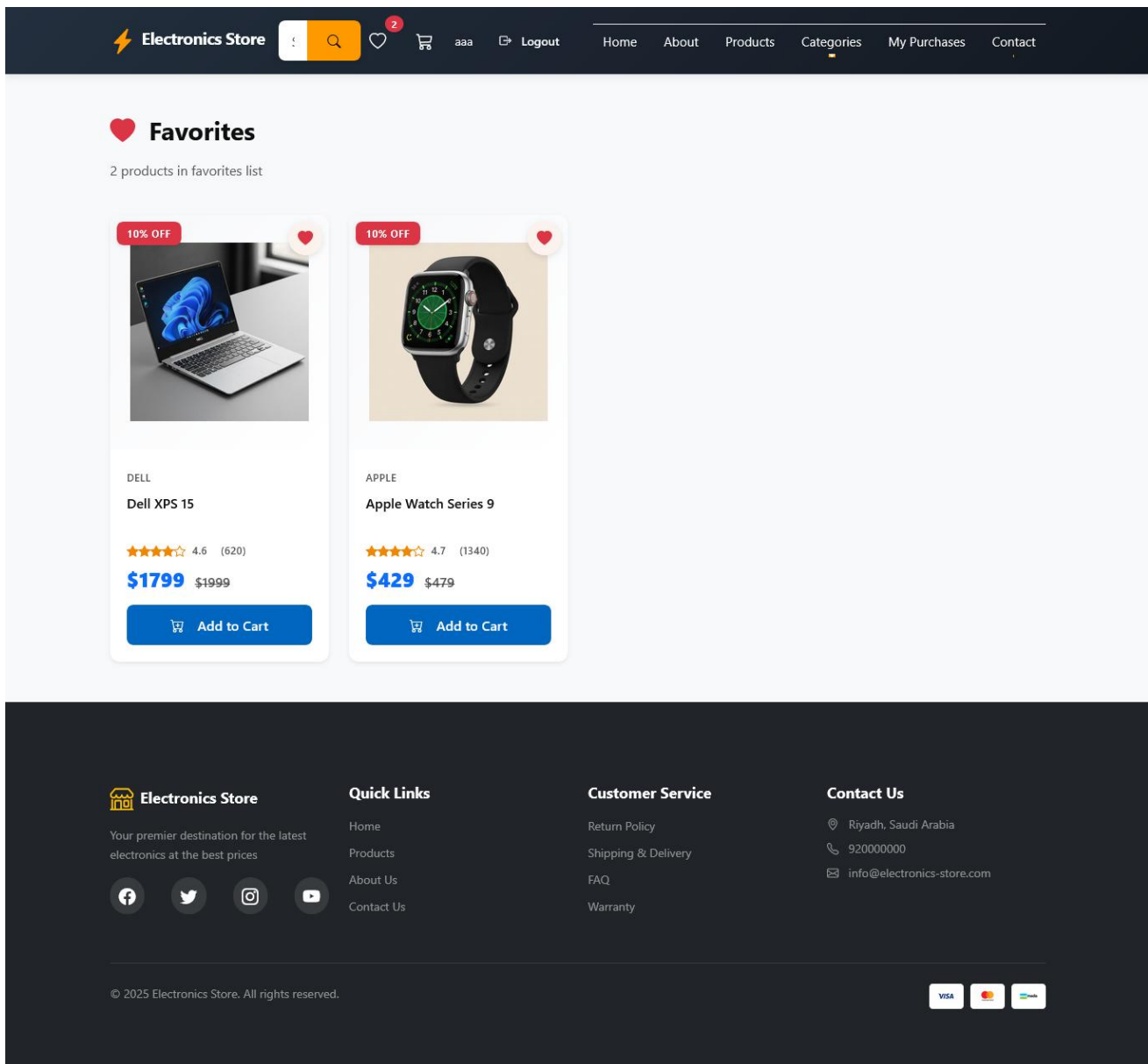


Figure 16: Favorites list with saved products.

The favorites page displays products saved by the customer for later. This improves the shopping experience by allowing users to return to preferred items without searching again.

Chapter 6

Testing

Chapter 6

Testing

Testing was done during development and after each sprint to check that all features work correctly. The team focused on the main workflows of registration, login, product browsing, checkout, Stripe payment, webhook confirmation, cancellation, and purchase history.

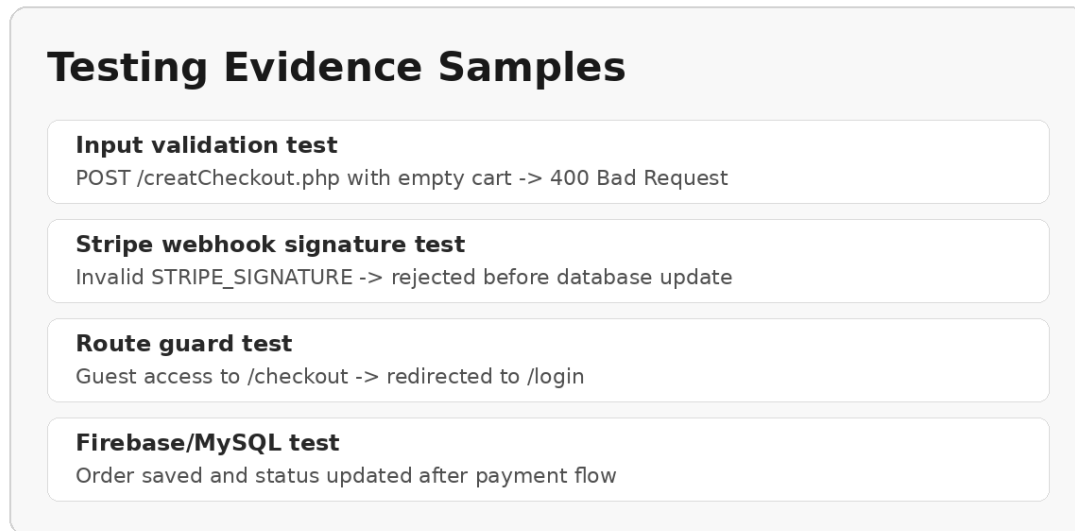


Figure 17: Testing evidence samples for validation, webhook, route guard, and order status checks.

Example webhook test logic:

```
$event = \Stripe\Webhook::constructEvent($payload, $signature, STRIPE_WEBHOOK_SECRET);
if ($event->type === 'checkout.session.completed') {
    // update order status to paid
}
```

6.1 Unit Testing

- Input validation — required fields checked in PHP and Vue.
- Firebase connection — product data loads correctly.
- MySQL queries — orders insert, update, and read correctly.
- Cart calculations — totals and quantities update correctly.
- Filter logic — product filters return correct results.
- Route guards — guests redirected to login from protected pages.

6.2 Integration Testing

- Cart to Stripe checkout — cart items appear correctly in Stripe line items.
- Firebase to PHP — product prices from Firebase appear correctly in Stripe session.
- Stripe webhook to MySQL — order status updates correctly after payment.
- COD/Bank/Card checkout — order saves correctly to Firebase (Buyers, Sales, My Purchases).
- My Purchases — orders saved during checkout appear in the My Purchases page.
- Cancel flow — cancelled order status updates in MySQL and cancel page displays correctly.

6.3 Functional Testing

Table 17: Test Cases

ID	Test Case	Expected Result	Status
TC01	Register with new email.	Account saved to Firebase. Redirect to login.	Pass
TC02	Register with existing email.	Error message shown.	Pass
TC03	Login with correct credentials.	User logged in. Redirect to home.	Pass
TC04	Login with wrong password.	Error message shown.	Pass
TC05	Search for a product by name.	Matching products appear.	Pass
TC06	Filter products by category.	Only products in that category shown.	Pass
TC07	Add product to cart.	Product appears in cart. Badge count updates.	Pass
TC08	Update cart quantity.	Cart total recalculated correctly.	Pass
TC09	Add product to favorites.	Product appears in favorites page.	Pass
TC10	Checkout with COD payment.	Order saved to Firebase. Cart cleared.	Pass
TC11	Checkout with Stripe payment.	Stripe session created. Redirect to Stripe page.	Pass
TC12	Complete Stripe payment.	Webhook received. Order status updated to paid.	Pass
TC13	Cancel Stripe payment.	Order status updated to cancelled in MySQL.	Pass
TC14	View My Purchases.	Orders filtered by user email appear correctly.	Pass
TC15	Refresh page — cart preserved.	Cart items still present from localStorage.	Pass
TC16	Guest tries to access /checkout.	Redirected to /login.	Pass
TC17	Send contact form.	Message saved to Firebase /Messages.	Pass

6.4 Non-functional Testing

No questionnaire-based user survey was conducted in this version. Therefore, non-functional testing was based on a structured manual review by the research team, focusing on usability, performance, security, and compatibility. Future evaluation can include a user questionnaire to collect wider feedback and recommendations.

Usability

All pages were reviewed to confirm they are clean and easy to use. Product cards, cart controls, and checkout forms were checked for clarity across different screen sizes.

Performance

Products load instantly because they are stored in the Pinia store in memory. Firebase reads for My Purchases and Messages are fast. The Stripe session creation takes 1-2 seconds which is normal for an API call.

Security

Stripe webhook signature verification was tested by sending requests with an invalid signature. PHP correctly rejected them with a 400 error. Route guards were tested by trying to access protected pages without authentication.

Compatibility

The website was tested on Chrome, Firefox, and mobile browsers. Bootstrap responsive classes worked correctly on desktop, tablet, and mobile screens.

6.5 Testing Results

All 17 test cases passed. The main workflows — registration, login, product browsing, cart, all four payment methods, Stripe webhook, order cancellation, and order history — work correctly. Small UI issues found during testing were fixed before the final sprint.

Chapter 7

Conclusion and Future Work

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project built a complete web-based electronic store. Customers can register, log in, browse products by category and brand, manage a shopping cart, save favorites, and pay using four different methods. Order history is available in the My Purchases page. The Stripe payment flow is confirmed reliably through webhooks.

The system uses Vue 3 with Pinia for the frontend, plain PHP files for the backend API, Firebase Realtime Database for product and user data, MySQL for Stripe order records, and Stripe for payment processing. Each part of the system is kept simple and focused. Each PHP file handles one job. Each Pinia store manages one type of data.

The project shows how to connect different technologies in one working system. All main software engineering steps were followed: requirements analysis, database design, implementation, and testing. The result is a working e-commerce application that can be extended in the future.

7.2 Limitations

- The product catalog is defined in the frontend application. Adding or editing products requires changing the code.
- The system supports English only.
- Delivery tracking and shipping cost calculation are not included.
- There is no analytics or sales reporting for the store owner.

7.3 Future Work

- Move the product catalog to Firebase so products can be added and managed without changing the code.
- Build an admin panel to manage products, view orders, and see sales reports.
- Add Arabic language support.
- Build native Android and iOS apps connected to the same backend.
- Integrate delivery companies to calculate shipping cost automatically.
- Add product reviews and ratings.
- Add discount coupons and promo codes.
- Add product recommendations based on purchase history.
- Improve security by migrating to Firebase Authentication SDK.
- Add stock alert notifications for low inventory.
- Integrate the Jawwal Pay API to support local digital wallet payments commonly used in Palestine.
- Integrate the Bank of Palestine (BOC/Bank of Palestine) payment API to enable direct online banking and local payment services.
- Replace the Stripe Sandbox environment with live Stripe production keys after completing security review and production deployment.

References

- [1] K. C. Laudon and C. G. Traver, E-commerce: Business, Technology, Society. Pearson, 2023. Access Date: 12/05/2026.
- [2] Vue.js, "Vue 3 Guide." Available: <https://vuejs.org/guide/>. Access Date: 12/05/2026.
- [3] Pinia, "Pinia Documentation." Available: <https://pinia.vuejs.org/>. Access Date: 12/05/2026.
- [4] PHP, "PHP Manual." Available: <https://www.php.net/manual/en/>. Access Date: 12/05/2026.
- [5] Stripe, "Stripe Checkout Documentation." Available: <https://stripe.com/docs/payments/checkout>. Access Date: 12/05/2026.
- [6] Firebase, "Firebase Realtime Database Documentation." Available: <https://firebase.google.com/docs/database>. Access Date: 12/05/2026.
- [7] Oracle, "MySQL 8.0 Reference Manual." Available: <https://dev.mysql.com/doc/refman/8.0/en/>. Access Date: 12/05/2026.
- [8] Bootstrap, "Bootstrap 5 Documentation." Available: <https://getbootstrap.com/docs/5.3/>. Access Date: 12/05/2026.
- [9] OWASP Foundation, "OWASP Top Ten." Available: <https://owasp.org/www-project-top-ten/>. Access Date: 12/05/2026.
- [10] K. Schwaber and J. Sutherland, "The Scrum Guide," 2020. Available: <https://scrumguides.org/>. Access Date: 12/05/2026.
- [11] Vue Router, "Vue Router 4 Documentation." Available: <https://router.vuejs.org/>. Access Date: 12/05/2026.
- [12] Netlify, "Netlify Documentation." Available: <https://docs.netlify.com/>. Access Date: 12/05/2026.
- [13] Amazon, "Amazon Online Shopping." Available: <https://www.amazon.com/>. Access Date: 12/05/2026.
- [14] Newegg, "Newegg Official Website." Available: <https://www.newegg.com/>. Access Date: 12/05/2026.
- [15] AliExpress, "AliExpress Official Website." Available: <https://www.aliexpress.com/>. Access Date: 12/05/2026.
- [16] Bootstrap Icons, "Bootstrap Icons Documentation." Available: <https://icons.getbootstrap.com/>. Access Date: 12/05/2026.
- [17] Apache Friends, "XAMPP Documentation." Available: <https://www.apachefriends.org/>. Access Date: 12/05/2026.
- [18] GitHub Docs, "GitHub Documentation." Available: <https://docs.github.com/>. Access Date: 12/05/2026.
- [19] Microsoft, "Visual Studio Code Documentation." Available: <https://code.visualstudio.com/docs>. Access Date: 12/05/2026.

[20] Postman, "Postman Learning Center." Available: <https://learning.postman.com/>. Access Date: 12/05/2026.

[21] Hostinger, "Hostinger Knowledge Base." Available: <https://support.hostinger.com/>. Access Date: 12/05/2026.

[22] Composer, "Composer Documentation." Available: <https://getcomposer.org/doc/>. Access Date: 12/05/2026.

[23] npm, "npm Documentation." Available: <https://docs.npmjs.com/>. Access Date: 12/05/2026.

Appendix

Appendix A: Customer Guide

How to use the electronic store:

1. Open the website in a browser.
2. Click Sign Up to create a new account. Enter your name, email, and password.
3. Log in with your email and password.
4. Browse products from the Products page. Use filters to narrow down by category, brand, or price.
5. Click on any product to see full details and specifications.
6. Click Add to Cart to add the product. Click the heart icon to save to favorites.
7. Open the Cart page to review items and quantities.
8. Click Checkout. Fill in your shipping information.
9. Select your payment method: Cash on Delivery, Credit Card, Bank Transfer, or Stripe.
10. For Stripe: you will be redirected to a secure Stripe payment page. Enter your card details there.
11. After payment, open My Purchases to see your order history.

Appendix B: Backend Environment Variables

```
# Stripe

STRIPE_SECRET_KEY=sk_test_...

STRIPE_PUBLIC_KEY=pk_test_...

STRIPE_WEBHOOK_SECRET=whsec_...


# Firebase

FIREBASE_PROJECT_ID=electronics-store-c9093

FIREBASE_API_KEY=...


# MySQL

DB_HOST=localhost

DB_NAME=u183914205_storProject

DB_USER=u183914205_storProject_u

DB_PASS=...


# URLs

SUCCESS_URL=https://electronics-store-project.netlify.app/stripe-success
```

```
CANCEL_URL=https://electronics-store-project.netlify.app/stripe-cancel  
FRONTEND_ORIGIN=https://electronics-store-project.netlify.app
```

Appendix C: Netlify Deployment Fix

The following file must exist at `public/_redirects` in the Vue project to prevent 404 errors on direct URL access:

```
/*      /index.html      200
```

This tells Netlify to serve Vue's `index.html` for all routes. Vue Router then handles the navigation. Without this file, URLs like `/stripe-success` and `/my-purchases` return a 404 page.